

Security Audit

Best in slot (Blockchain infrastructure)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	38
Conclusion	39
Our Methodology	40
Disclaimers	42
About Hashlock	43

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Best in slot team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Best in Slot (BiS) is the lead maintainer of the BRC20 token protocol on Bitcoin and architect of the BRC2.0 upgrade, enabling programmable Bitcoin-native assets and EVM smart contracts on Bitcoin L1. Beyond protocol stewardship, BiS powers +100 projects with comprehensive APIs for BRC20, Ordinals, and Runes. The team also builds consumer-facing products — including BiS Explorer, BiS DEX, and CoinCooker.com launchpad, driving Bitcoin's DeFi and cultural ecosystem forward.

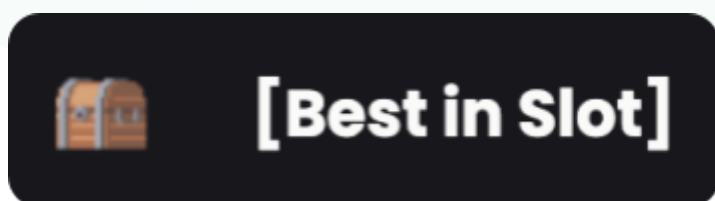
Project Name: Best in slot

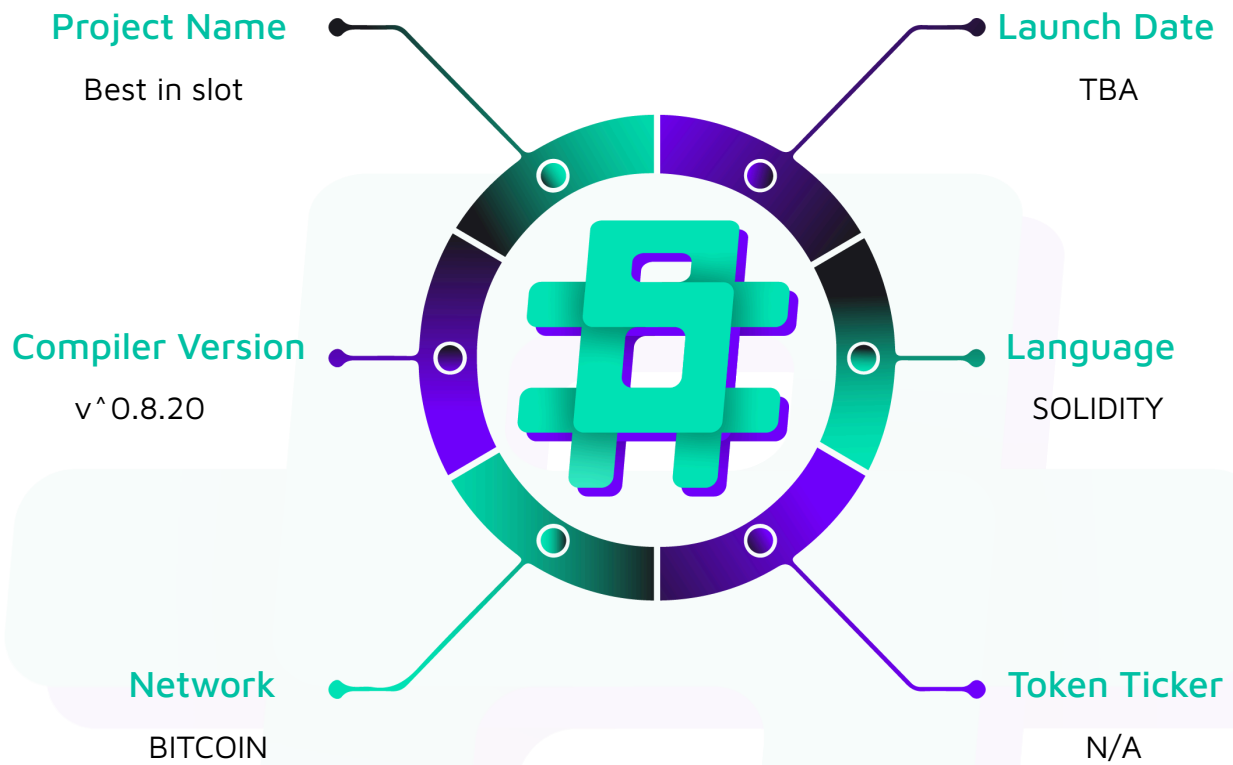
Project Type: Blockchain infrastructure

Compiler Version: ^0.8.20

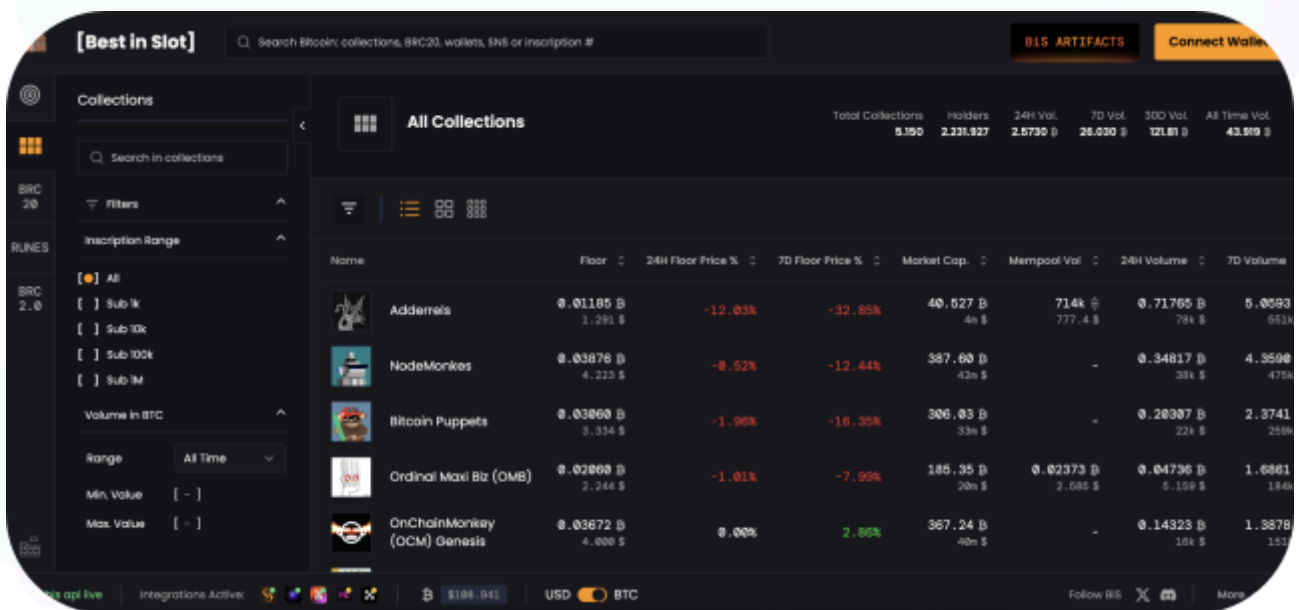
Website: <https://bestinslot.xyz/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Best in slot project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Best in slot Smart Contracts
Platform	Bitcoin BRC2.0 / Solidity
Audit Date	September, 2025
Contract 1	CoinCookerServer.sol
Contract 1 MD5 Hash	a57388996d0d02ffdd0e764babca4124
Contract 2	l1-orderbook.sol
Contract 2 MD5 Hash	de4995ce55e77f54a32e9aa0e863c8f3
Fix Review GitHub Commit Hash	a007c68487df14eeb6632d34236ec5a6e85ad87c

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 Medium severity vulnerabilities

14 Low severity vulnerabilities

1 Gas Optimisation

3 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
I1-orderbook.sol Implements a decentralized orderbook for trading ERC-20 tokens against Bitcoin (BTC) on a Bitcoin Layer 1 solution. It enables users to create sell orders for tokens with BTC pricing, and buyers can purchase tokens using Bitcoin payments verified through BIP322 signatures and Bitcoin transaction validation. Allows users to: - Create sell orders by depositing tokens and setting the BTC price - Cancel their own sell orders and get the remaining tokens back - Buy tokens from sell orders using Bitcoin payments - Request refunds for Bitcoin payments Allow admin to: - Set protocol params - Set trusted contract addresses - Enable/Disable all buy functionality - Enable/Disable all sell functionality	Contract achieves this functionality.
CoinCookerServer.sol Manages token distribution through multiple mechanisms: Merkle tree-based airdrops, linear vesting schedules, and direct airdrops. Allows users to: - Claim tokens using Merkle proofs - Claim vested tokens according to their schedule	Contract achieves this functionality.

- Check their vesting schedule details and claimable amounts

Allows admin to:

- Set Merkle root for new airdrop rounds
- Control when merkle tree-based claims are enabled
- Add vesting schedules
- Change vesting addresses
- Execute direct airdrops to multiple addresses
- Withdraw any ERC-20 tokens from the contract

Code Quality

This audit scope involves the smart contracts of the Best in slot project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Best in slot project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] 11-orderbook, CoinCookerServer - Unsafe token transfers using direct transfer methods instead of SafeERC20

Description

The contracts use direct `IERC20.transfer()` and `IERC20.transferFrom()` methods instead of OpenZeppelin's SafeERC20 library, which can lead to silent failures with non-standard ERC20 tokens.

Vulnerability Details

The codebase uses direct calls to `IERC20.transfer()` and `IERC20.transferFrom()` methods without proper error handling for non-standard ERC20 tokens. While some calls include `require()` statements to check return values, this approach is not sufficient for all token implementations, as some ERC20 tokens do not return boolean values.

This affects the transfers in the following methods of the `Claim` contract:

- `claimMerkle()`
- `claimVesting()`
- `withdrawEmergency()`
- `airdrop()`

Also affects the transfers in the following methods of the `11-orderbook` contract:

- `create_sell_order()`
- `cancel_sell_order()`
- `_buy_tokens_inner()`

Impact

Non-standard ERC20 tokens may cause transactions to fail unexpectedly or behave inconsistently, potentially breaking core functionality of the contracts.

Recommendation

Replace all direct `transfer()` and `transferFrom()` calls with OpenZeppelin's SafeERC20 library methods. Add the following import and usage:

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

contract /* */ {

    using SafeERC20 for IERC20;

    // Replace IERC20(token).transfer() → token.safeTransfer()

    // Replace IERC20(token).transferFrom() → IERC20(token).safeTransferFrom()

}
```

Status

Resolved

[M-02] I1-orderbook, CoinCookerServer - Centralization risk**Description**

Both contracts use a single owner address with extensive administrative privileges, creating significant centralization risks that could lead to loss of funds, denial of service, or manipulation of critical parameters.

Vulnerability Details

The contracts implement a single owner address that has control over critical functions controlling the protocol parameters, token claims, and trading.

Impact

A compromised or malicious owner can cause direct loss of funds, denial of service, or manipulation of the protocol's core functionality, affecting all users of the system.

Recommendation

Use a multisig wallet for the address controlling the owner role. This would require multiple approvals for critical actions

Status

Acknowledged

[M-03] I1-orderbook#buy_tokens, I1-orderbook#buy_tokens_trusted -

Missing slippage protection allows users to receive fewer tokens than expected

Description

The `buy_tokens` and `buy_tokens_trusted` functions lack slippage protection mechanisms. Users can receive fewer tokens than expected due to partial fills or insufficient order liquidity, without any ability to specify minimum amounts or revert the transaction.

Vulnerability Details

In the `_buy_tokens_inner` function, users specify desired amounts for each order in the `amounts` array, but there's no validation to ensure they receive at least a minimum amount of tokens. The `_process_order` function can reduce the actual filled amount due to insufficient liquidity if the order was partially filled by other users or order rejection due to minimum price constraints.

The transaction continues to execute even when users receive fewer tokens than requested, and there's no mechanism for users to specify minimum acceptable amounts or revert the transaction when slippage exceeds their tolerance.

Impact

Users may receive significantly fewer tokens than expected

Recommendation

Implement slippage protection by adding a `min_amounts` parameter that allows users to specify minimum acceptable amounts for each order. The function should revert if any order results in fewer tokens than the specified minimum.

Note

The Best in Slot team informed Hashlock that this product is designed as an order book, not a swap mechanism. Order matching is handled by the sequencer, which also generates the `buy_signature`. As a result, orders are highly likely to be filled. Even in cases where they are not, the execution price will remain unchanged. Unlike automated market maker (AMM) swaps—where transaction ordering in the mempool can cause significant price shifts—this system does not include auto-matching logic in the smart

contract. Therefore, while users may occasionally receive fewer tokens due to partial fills, they will always receive them at the price they initially agreed to.

Status

Acknowledged



Low

[L-01] I1-orderbook, CoinCookerServer - Floating pragma

Description

Both contracts use floating pragma statements `pragma solidity ^0.8.20`, instead of fixed version pragmas.

Vulnerability Details

Floating pragmas allow contracts to be compiled with different compiler versions than those tested, potentially introducing unidentified security issues.

Impact

Different pragma versions in test and mainnet may pose unidentified security issues.

Recommendation

Consider locking the pragma version to the specific version `pragma solidity 0.8.20`.

Status

Resolved

[L-02] I1-orderbook, CoinCookerServer - Missing events for admin parameter changes

Description

Admin functions that modify protocol parameters do not emit events, making it difficult to track important configuration changes and reducing transparency for users and external systems

Vulnerability Details

The `I1-orderbook` contract contains multiple admin functions that modify protocol parameters without emitting corresponding events.

These functions include:

- `constructor()`
- `setOrderSignerWallet()`
- `setHandlerPkscriptHash()`
- `setServiceFeeDustLimit()`
- `setMinSellBtcPriceSats()`
- `setMaxSellBtcPriceSats()`
- `setMaxSellTokenAmount()`
- `setTrustedBuyContract()`
- `disableTrustedBuyContract()`
- `disableBuys()`
- `disableSells()`
- `enableSells()`

Additionally, the `Claim` contract has an emergency withdrawal function that also lacks event emission:

- `withdrawEmergency()`

Impact

Lack of transparency and auditability for protocol parameter changes, making it difficult to track and verify admin actions.

Recommendation

Add appropriate events for all the functions listed above.

Status

Resolved

[L-03] `l1-orderbook#create_sell_order` - Fee on transfer tokens not supported

Description

The `create_sell_order` function transfers tokens from the user to the contract but doesn't account for fee on transfer tokens, which can cause accounting discrepancies between the expected and actual token amounts received.

Vulnerability Details

The contract stores the `token_amount` parameter as the `remaining_token_amount` in the sell order, assuming this is the exact amount received. However, some ERC20 tokens implement a fee on transfer mechanisms where a percentage of the transferred amount is taken as a fee, meaning the actual amount received by the contract is less than the `token_amount` specified.

Impact

Fee on transfer tokens can cause sell orders to be created with incorrect accounting, potentially leading to failed token transfers when buyers attempt to purchase tokens.

Recommendation

Consider either documenting that fee on transfer tokens are not supported or supporting fee on transfer tokens by checking the contract balance before and after the `transferFrom` call.

Status

Resolved

[L-04] I1-orderbook#_buy_tokens_inner - Signature malleability vulnerability in ecrecover implementation

Description

The contract uses a custom `ecrecover` implementation that is vulnerable to signature malleability attacks, which could allow attackers to replay signatures with modified parameters.

Vulnerability Details

The contract implements a custom signature verification mechanism in the `_buy_tokens_inner` function that directly uses Solidity's `ecrecover()` function. This implementation is vulnerable to signature malleability because the elliptic curve used in Ethereum signatures is symmetrical, meaning for every valid signature $[v,r,s]$, there exists another valid signature $[v,r,s]$ that produces the same result.

```

uint8 v;

bytes32 r;

bytes32 s;

require(buy_signature.length == 65, "Invalid signature length");


bytes memory signature = buy_signature;

assembly {

    r := mload(add(signature, 32))

    s := mload(add(signature, 64))

    v := byte(0, mload(add(signature, 96)))

}


require(v == 27 || v == 28, "Invalid signature v value");

require(r != bytes32(0) && s != bytes32(0), "Invalid signature r or s value");


bytes32 message_hash = keccak256(message);

address signer = ecrecover(message_hash, v, r, s);

require(signer != address(0) && signer == order_signer_wallet, "Invalid signature");

```

While the contract does include some basic validation (checking v values and non-zero r/s), it doesn't implement the necessary checks to prevent signature malleability. An attacker could potentially compute an alternative valid signature for the same message and use it to replay transactions.

Impact

Signature malleability could allow attackers to replay signatures with modified parameters, potentially bypassing intended security controls and authorization mechanisms.

Recommendation

Replace the custom ecrecover implementation with OpenZeppelin's ECDSA library, which includes built-in protection against signature malleability.

```
import {ECDSA} from "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";

// Replace the custom signature verification with:

address signer = ECDSA.recover(keccak256(message), buy_signature);

require(signer != address(0) && signer == order_signer_wallet, "Invalid signature");
```

Status

Resolved

[L-05] I1-orderbook#_buy_tokens_inner - Zero value transfer may revert with non-standard ERC20 tokens

Description

The contract performs token transfers without checking if the transfer amount is greater than zero, which can cause transactions to revert when interacting with ERC20 tokens that revert on zero-value transfers.

Vulnerability Details

The `_buy_tokens_inner` function directly calls `IERC20(order.token).transfer(buyer, filled_amounts[i])` without first checking if `filled_amounts[i] > 0`. The `filled_amounts` array is populated based on the order execution logic, and it's possible for some elements to be zero in certain scenarios.

Some ERC20 tokens are known to revert when attempting to transfer zero amounts can cause the entire transaction to fail.

Impact

Transactions may revert unexpectedly when the contract interacts with ERC20 tokens that implement zero-value transfer restrictions.

Recommendation

Add a check to only perform the transfer when the amount is greater than zero.

Status

Resolved

[L-06] I1-orderbook#setServiceFeeDustLimit - Missing maximum validation allows owner to prevent service fee collection

Description

The `setServiceFeeDustLimit` function lacks a maximum value validation, allowing the owner to set the dust limit so high that service fees are never collected, resulting in loss of revenue.

Vulnerability Details

The `setServiceFeeDustLimit` function only validates that the new value is greater than 0, but does not enforce an upper bound.

The service fee collection logic in the `_buy_tokens_inner` function only collects fees when the calculated service fee is greater than or equal to the dust limit.

If the owner sets `service_fee_dust_limit` to a high value the condition `service_fee >= service_fee_dust_limit` will never be true, effectively disabling service fee collection for all transactions.

Impact

The owner can permanently disable service fee collection by setting an excessively high dust limit, resulting in a complete loss of revenue from service fees.

Recommendation

Add a maximum validation to the `setServiceFeeDustLimit` function to prevent the dust limit from being set to unreasonably high values. Consider setting a maximum based on typical transaction sizes or a reasonable percentage of the maximum possible service fee.

Status

Acknowledged

[L-07] I1-orderbook#setMinSellBtcPriceSats/setMaxSellBtcPriceSats -

Missing validation between min and max sell BTC price sats

Description

The contract lacks validation to ensure that `min_sell_btc_price_sats` is always lower than `max_sell_btc_price_sats` in both the constructor and the setter functions.

Vulnerability Details

In the constructor, the contract sets both `min_sell_btc_price_sats` and `max_sell_btc_price_sats` without validating that the minimum is lower than the maximum.

Similarly, in the setter functions, both `setMinSellBtcPriceSats` and `setMaxSellBtcPriceSats` only validate that the new values are positive but do not check the relationship between them.

If `min_sell_btc_price_sats` is set higher than `max_sell_btc_price_sats`, it becomes impossible to create any sell orders, effectively breaking the core functionality of the contract.

Impact

If the minimum sell BTC price is set higher than the maximum sell BTC price, all sell order creation will be blocked, breaking the primary functionality of the orderbook contract.

Recommendation

Add validation in both the constructor and setter functions to ensure `min_sell_btc_price_sats` is always lower than `max_sell_btc_price_sats`.

Status

Resolved

[L-08] `l1-orderbook#create_sell_order` - Max sell token amount validation doesn't account for token decimals

Description

The `create_sell_order` function validates that `token_amount` `<=` `max_sell_token_amount`, but this validation doesn't consider that different tokens have different decimal places (6, 8, 18 decimals), which can lead to inconsistent validation across different token types.

Vulnerability Details

The `max_sell_token_amount` is a single value that applies to all tokens regardless of their decimal places. This means:

- For a token with 18 decimals (like most ERC20 tokens), the validation works as expected
- For a token with 6 decimals (like USDC), the same `max_sell_token_amount` value represents 10^{12} times more actual tokens
- For a token with 8 decimals (like some stablecoins), the same value represents 10^{10} times more actual tokens

Impact

The validation creates inconsistent limits leading to potentially unintended large sell orders for tokens with fewer decimals.

Recommendation

Consider normalizing the validation based on token decimals.

Status

Acknowledged

[L-09] `l1-orderbook#_buy_tokens_inner` - Missing validation for `bip322_signature` length

Description

The `_buy_tokens_inner` function accepts a `bip322_signature` parameter but does not validate that its length is greater than 0, unlike other parameters in the same function, which have proper validation.

Vulnerability Details

In the `_buy_tokens_inner` function, the `bip322_signature` parameter has no validation to ensure that the signature has a length greater than 0. This is inconsistent with the validation pattern used for other parameters in the same and other functions.

The function later uses this signature in the `_BIP322Verify` call, but without proper length validation, it could potentially accept empty signatures, which may lead to unexpected behavior.

Impact

The missing validation creates inconsistency in the codebase and deviates from the established validation pattern, potentially leading to unexpected behavior.

Recommendation

Add a validation check for `bip322_signature.length > 0`

Status

Resolved

[L-10] I1-orderbook#request_refund_trusted - Missing parameter validation in trusted function**Description**

The `request_refund_trusted` function lacks parameter validation that is present in the public `request_refund` function, potentially allowing invalid parameters to be processed.

Vulnerability Details

The `request_refund` function performs parameter validations before calling `_request_refund_inner`.

However, the `request_refund_trusted` function skips these validations and directly calls `_request_refund_inner` with the provided parameters. While the function is restricted to the trusted buy contract, it should still validate parameters to ensure data integrity and prevent potential issues.

Impact

Missing parameter validation in the trusted function could lead to the processing of invalid data, potentially causing unexpected behavior.

Recommendation

Consider moving the parameter validation to the `_request_refund_inner` function.

Status

Resolved

[L-11] l1-orderbook#_buy_tokens_inner - No maximum validation for `service_fee_percentage` and `extra_btc_fee_for_handler` leading to potential excessive fee extraction

Description

The `service_fee_percentage` and `extra_btc_fee_for_handler` parameters lack proper validation, allowing them to exceed reasonable limits and potentially charge users excessive fees that could exceed the transaction value itself.

Vulnerability Details

Both `service_fee_percentage` and `extra_btc_fee_for_handler` parameters are used in `_buy_tokens_inner` to calculate total costs and fees. However, there is no validation to ensure these parameters are within reasonable bounds. The parameters can be set to any arbitrary values, including values that would result in fees exceeding 100% of the transaction cost.

Even if the user signed the buy message with the correct parameters, having no maximum validation opens the door to potential exploitation if the user is not aware of the fees charged or is tricked into paying higher fees.

Impact

Users can be charged excessive fees that may exceed the transaction value itself through either excessive service fee percentages or excessive handler fees.

Recommendation

Implement proper validation for both parameters by adding maximum limit checks. Consider adding maximum limits and validating the parameters before using them in calculations:

```
require(service_fee_percentage <= MAX_SERVICE_FEE_PERCENTAGE, "Service fee percentage exceeds maximum allowed");

require(extra_btc_fee_for_handler <= MAX_EXTRA_BTC_FEE, "Extra BTC fee for handler exceeds maximum allowed");
```

Additionally, consider adding events to the log when fees are charged and providing transparency to users about the fee structure.

Status

Acknowledged

[L-12] l1-orderbook - Owner parameter changes lack safeguards for existing transactions

Description

The `sethandlerPkscriptHash` and `setOrderSignerWallet` functions allow the owner to change critical parameters without any safeguards to ensure existing transactions are processed first, potentially causing temporary operational issues and invalidating pending operations.

Vulnerability Details

The `handler_pkscript_hash` parameter is used in critical validation checks within the `_buy_tokens_inner` and `_request_refund_inner` functions to verify that Bitcoin transactions were sent to the correct handler address. When this parameter is changed via the `sethandlerPkscriptHash` function, all existing Bitcoin transactions sent to the previous handler address become invalid.

The `order_signer_wallet` parameter is used to verify buy transaction signatures in the `_buy_tokens_inner` function. When this parameter is changed via the `setOrderSignerWallet` function, all previously signed buy messages become invalid because the signature verification checks against the new signer wallet address.

Impact

Changing these parameters without safeguards can cause temporary operational issues for users with pending transactions, invalidate Bitcoin payments sent to the old handler address, and prevent signed buy messages from being executable. Although the situations are reversible by the owner, they can cause immediate disruption to ongoing operations.

Recommendation

Consider implementing safeguards for both parameter changes by handling both old and new parameter values during a transition period, adding a delay before parameter changes take effect, or requiring all pending transactions to be processed before allowing parameter changes.

Status

Acknowledged

[L-13] I1-orderbook#setMinSellBtcPriceSats - `min_sell_btc_price_sats` increase may make existing orders permanently unfillable

Description

When the `min_sell_btc_price_sats` is increased, existing orders (both partially filled and unfilled) that were created with a price below the new minimum or the remaining tokens' cost falls below the new minimum may become permanently unfillable due to price validation checks.

Vulnerability Details

The contract allows the owner to update `min_sell_btc_price_sats` through the `setMinSellBtcPriceSats()` function without any consideration for existing orders. When processing buy orders in `_process_order()`, the contract validates that the order's cost meets the current minimum price:

```
uint256 cost = (amount * order.total_btc_price_sats) / order.token_amount;

if (amount > 0) {

    if (cost < min_sell_btc_price_sats) {

        // cannot sell below min_sell_price (NOTE: this should never happen since we also
        check for remaining balance btc amount)

        // reject this buy order

        order.remaining_token_amount += amount; // revert

        amount = 0;
```

```

        cost = 0;

    }

}

```

Additionally, for partially filled orders, there's a second validation check:

```

if (order.remaining_token_amount > 0) {

    uint256    remaining_tokens_cost    =    (order.remaining_token_amount    *
order.total_btc_price_sats) / order.token_amount;

    if (remaining_tokens_cost < min_sell_btc_price_sats) {

        // cannot sell remaining tokens below min_sell_price so if we fullfil this buy,
remaining tokens will be locked

        // reject this buy order

        order.remaining_token_amount += amount; // revert

        amount = 0;

        cost = 0;

    }

}

```

While this validation check initially may pass, after increasing the minimum sell price, if there is a new buy for the same order for the remaining tokens, the cost is calculated again, and if it falls below the new minimum price, the order becomes unfillable.

Impact

Existing orders whose cost falls below the new minimum price become permanently unfillable, preventing legitimate trades and forcing sellers to cancel their orders to recover remaining tokens.

Recommendation

Before increasing `min_sell_btc_price_sats`, carefully review all existing orders to identify those that would become unfillable. Prevent submission of new buy orders for

affected orders and notify the order owners to cancel their orders to recover remaining tokens.

Status

Acknowledged

[L-14] l1-orderbook#create_sell_order - Bitcoin address validation only checks for non-empty string without proper format validation

Description

The `create_sell_order` function validates that the `btc_recv_addr` variable is not empty, but fails to validate the proper Bitcoin address format and length, which could lead to invalid addresses being accepted.

Vulnerability Details

In the `create_sell_order` function, the `btc_recv_addr` variable validation is insufficient because it only ensures the address is not empty, but doesn't validate the proper Bitcoin address format or length. The current validation allows any non-empty string to be accepted as a Bitcoin address, which could lead to invalid addresses being stored.

Impact

Invalid Bitcoin addresses could be accepted, potentially leading sellers not to receive their funds in the Bitcoin network.

Recommendation

Implement proper Bitcoin address validation that, at a minimum, validates the address length with appropriate limits.

Status

Acknowledged

Gas

[G-01] CoinCookerServer#claimMerkle, CoinCookerServer#claimVesting - **nonReentrant** modifier unnecessary in current implementation

Description

The `claimMerkle` and `claimVesting` functions use the `nonReentrant` modifier, but this protection is unnecessary.

Vulnerability Details

The reentrancy protection is unnecessary because all state variables in `claimMerkle` and `claimVesting` are updated before the external call; also, the function follows the checks-effects-interactions pattern, so there is no impact on a potential reentrancy.

Impact

The unnecessary `nonReentrant` modifier adds gas overhead without providing security benefits in the current implementation.

Recommendation

Remove the `nonReentrant` modifier usage and update `Claim` to not extend the `ReentrancyGuard` contract.

Status

Resolved

QA

[Q-01] l1-orderbook, CoinCookerServer - Missing NatSpec documentation

Description

The contracts `Claim` and `l1-orderbook` lack NatSpec documentation that explains their intended functionality. This makes the code harder to understand, maintain, and audit.

Vulnerability Details

Both contracts lack NatSpec documentation for function purposes, parameters, return values, implementation details, events, and state variable purposes.

Impact

Lack of documentation makes the code harder to understand, maintain, and integrate with, while also reducing audit efficiency.

Recommendation

Add comprehensive NatSpec documentation to all public and external functions, events, and important state variables.

Status

Resolved

[Q-02] I1-orderbook - Hardcoded addresses and numbers should be constants

Description

The contract contains hardcoded addresses and magic numbers that should be defined as constants for better maintainability and code clarity.

Vulnerability Details

The contract contains several hardcoded values that should be defined as constants:

```
// Hardcoded precompile addresses:

0x00000000000000000000000000000000fd // in _getTxDetails()

0x00000000000000000000000000000000fe // in _BIP322Verify()

// Hardcoded magic number:

10000 // in service fee calculation (basis points divisor)
```

Impact

Hardcoded values reduce code maintainability and readability, potentially leading to errors during future modifications or deployments.

Recommendation

Define these values as constants at the contract level:

```
// Precompile addresses

address          internal          constant          BTC_TX_DETAILS_ADDRESS          =
0x00000000000000000000000000000000fd;

address          internal          constant          BIP322_VERIFY_ADDRESS          =
0x00000000000000000000000000000000fe;


uint256 internal constant BASIS_POINTS_DIVISOR = 10000;
```

Status

Resolved

[Q-03] I1-orderbook - Code duplication in vin validation logic**Description**

The same vin validation logic is duplicated in two different functions, which makes the code harder to maintain.

Vulnerability Details

The vin validation logic that checks transaction inputs and calculates total values is repeated in two locations, in the `_request_refund_inner` and `_buy_tokens_inner` functions.

The only difference between these two blocks is the variable name used for the required value (`refund_sats` and `total_expected_vin_value`).

Impact

Code duplication increases maintenance and the risk of introducing bugs when changes need to be made to the vin validation logic.

Recommendation

Extract the vin validation logic into a common internal function that takes the required value as a parameter.

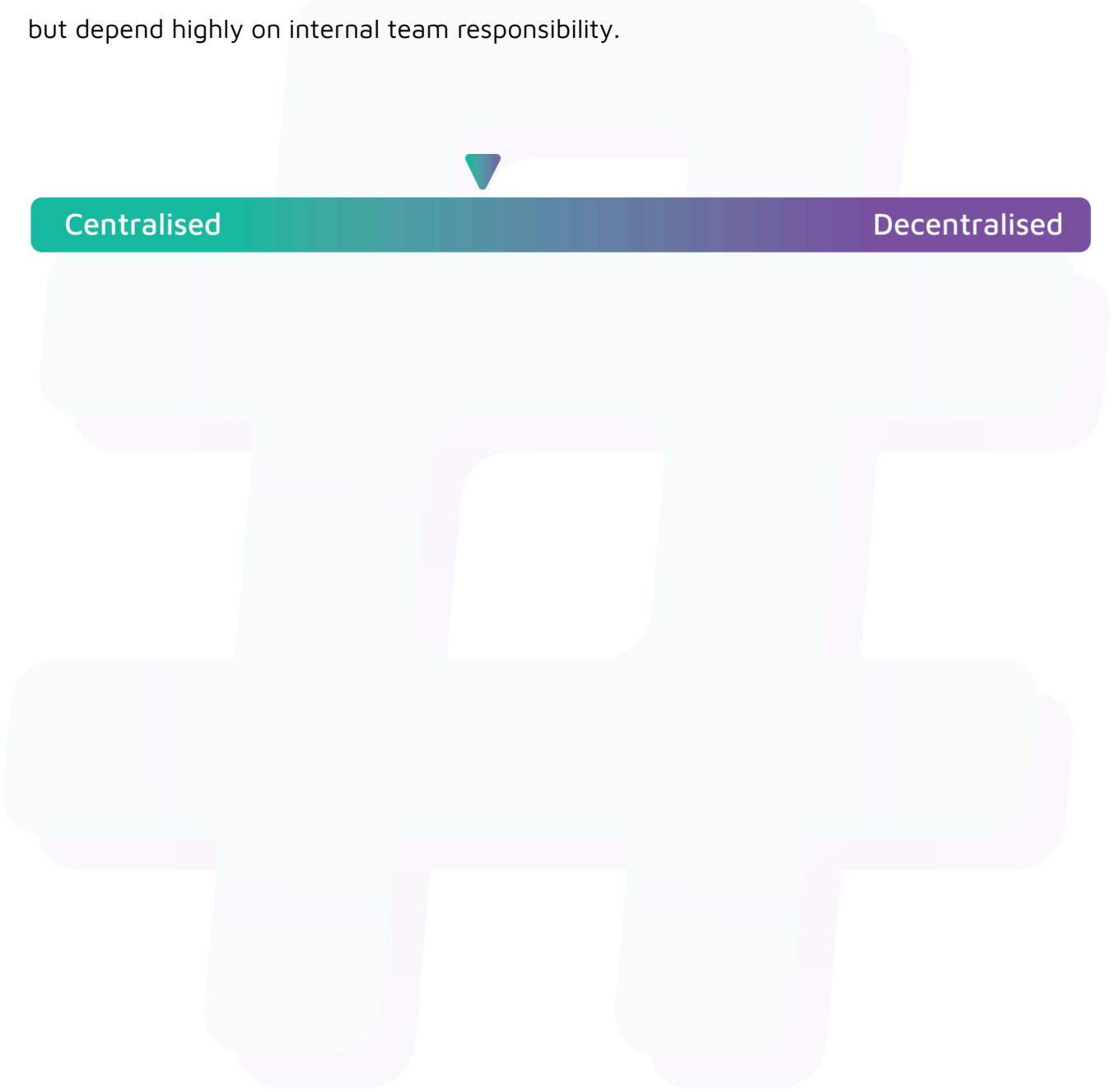
Status

Acknowledged

Centralisation

The Best in slot project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Best in slot project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.